International Journal of Innovation Management
Vol. 20, No. 1 (January 2016) 1650014 (28 pages)

DOI: 10.1142/S1363919616500146

Imperial College Press
www.icpress.co.uk

A COGNITIVE LOAD VIEW AND EMPIRICAL TEST OF COLLABORATION NETWORK STRUCTURE VERSUS LEARNING RATES IN NEW SOFTWARE DEVELOPMENT

JORGE COLAZO

Trinity University San Antonio, Texas, USA
jcolazo@trinity.edu

Published 22 June 2015

This study explores whether characteristics of the collaboration structure in software development teams affect development learning rates, with a secondary goal of testing product complexity as a moderator. We develop suitable hypotheses under the theoretical lens of cognitive load theory. The empirical study uses archival data on an ordinary least squares model to find significant associations between collaboration structure, product complexity and the learning rates exhibited by 230 development teams producing open source software. Results show two distinct subgroups of projects: The first subgroup exhibits an average 78% learning rate, and the other subgroup "unlearned", i.e., productivity deteriorated over time instead of improved. In the learning subgroup, collaboration network density negatively impacted learning, while product complexity interacted with collaboration network centralisation and boundary spanning activity. In the unlearning subgroup, only network density impacted learning rates and no moderating effects were found. Practical implications and future opportunities for research are discussed.

Keywords: Collaboration; open source software; software development; cognitive load; learning rates.

Introduction

With the obvious and growing importance of software for business and almost every other aspect of modern life, academia and practitioners endeavour to find better tools and techniques to make the new software project development process more efficient. Though software development project success and failure rates have been hotly argued (Glass, 2005), there is consensus in indicating that much remains to be improved and the development of new systems is far from efficient, with economic and societal consequences that are hugely important (Dwivedi *et al.*, 2014).

In facing programmer delays, cost overruns and programmer attrition (Stepanek, 2011), promoting a rapid increasing of the productivity (i.e., learning) among development teams appears as a critical priority. Disappointingly, learning in software development presents a high variability, with some development teams exhibiting a high rate of learning, and a steady productivity increase in time (Boh *et al.*, 2007), whilst other software development teams "not only fail to learn but learn to fail" (Lyytinen and Robey, 1999).

Factors impacting the learning rates of software development teams relate to the product being developed, contextual factors, and also to the structure of the team, the latter being the focus of this paper.

Extant literature has looked at the impact of team structure on development performance mostly from a reductionist approach that described such structure, for instance as being either hierarchical or bureaucratic, or possessing either a strong or weak leadership. This reductionist approach cannot capture key aspects of collaboration such as how many team members collaborate with others, whether one or few members concentrate collaboration while others work in isolation, and in general anything that has to do with relational aspects of collaboration. Such reductionist approach belongs to the tradition of "factor research" (Mohr, 1982) which, while useful, does not allow a standardised and straightforward comparison between teams and is difficult to translate into clear normative prescriptions (Dwivedi *et al.*, 2014). For instance, describing a team as "hierarchical" does not shed light on whether one or more team members centralise the collaboration effort, or to what degree the tasks are shared by team members.

Given that "IS success (or failure) is a direct consequence of the effectiveness of collaboration" (Dwivedi *et al.*, 2014: 8), a look at the inside structure of collaboration networks and their relationships with learning, including collaboration relationships among different teams, is warranted, which brings about the paper's first research question: *What kind of collaboration structure associates with higher learning rates in software development*?

Building on the rich tradition from educational psychology of looking at learning as a cognitive phenomenon, and responding to a call to link networks with cognition (Davern *et al.*, 2012), we are going to rely on Social Network Analysis (SNA) (Wasserman and Faust, 1994) to describe collaboration structures and on cognitive load theory (Sweller, 1988; Chandler and Sweller, 1991; Sweller, 1994; Kalyuga *et al.*, 1998; Sweller *et al.*, 1998; Kalyuga *et al.*, 2003; Sweller *et al.*, 2007) to hypothesise links between collaboration network structure and development learning rates.

Cognitive load theory indicates that product complexity should influence the intrinsic cognitive load of individuals and teams, potentially impacting learning rates. Consistently, product complexity is, in this paper, treated as a moderating

factor, and is the object of the second main research question for this study: *How does product complexity influence the relationship between collaboration structure and learning rates*?

With the main goal of understanding of how the structuring of new software development teams can affect their learning rates, after deriving a set of hypotheses, we then test those using archival data from a large sample of open source software (OSS) project teams. The analysis is performed applying ordinary least squares regression to network parameters including collaboration network density, centralisation, and boundary spanning activity (BSA).

Section "Literature Review and Hypotheses" presents the literature review and hypotheses development. Section "Methods" includes the analytical methods and section "Analysis and Results" presents the results of ordinary least squares models. The results are discussed in section "Discussion" while the study's limitations and implications for future research are laid out in section Limitations and Future Research", followed by references to previous research.

Literature Review and Hypotheses

First, we review how collaboration structure has been defined and specified in the literature and the evidence of an association between structure and performance in general, and learning in particular. Second, we look back on research on learning rates and what appears to influence them. Third, we review the main works that lay the theoretical foundation for the hypotheses to be tested: cognitive load theory. Last, drawing from the mentioned streams of literature we develop hypotheses to be empirically tested.

Collaboration structure

Collaboration refers, in this paper, to the joint work by team members on the same task. Collaboration structure, as any other kind of social structure, can be represented using concepts of SNA (Wasserman and Faust, 1994). Adherents to SNA see traits of social actors and their collectives as a result of the actors' embeddedness into different types of networks. A network is a set of relationships among social actors where the actors are nodes of a network-like structure and there is a relationship, or link, between two actors when a certain condition is met, such as, when they cooperate in the same task.

Teams or work groups can thus be characterised by looking at the properties of the networked structures to which they belong (Wasserman and Faust, 1994). The mathematics of graphs help us identify unique traits of those networks that define any given group and allow comparison with other groups.

Of the several network characteristics that can be calculated to identify a group of actors, network density is perhaps the best known. Density is defined as the ratio between the number of actually observed links between actors to the maximum possible number of links for the group size. Density ranges between zero and one, with zero meaning that all actors are isolates, and one meaning that all actors are linked (collaborate) with all other actors in the network. Density measures the interconnectedness or cohesiveness of a social structure. In valued networks (where each tie has a value or "strength"), density is the average value of tie strength standardised by the size of the network (Wasserman and Faust, 1994).

Some researchers have found a positive association between network density and team performance in new product development (Reagans and Zuckerman, 2001). Others argue that the relationship is inverse or U-shaped (Oh *et al.*, 2004; Chidambaran and Tung, 2005). For large teams and relatively complex tasks a negative association between density and performance was found (Balkundi and Harrison, 2006). Although there are few empirical studies specific to software development, the complexity of developing modern software suggests that we should observe a negative relationship, at least for relatively larger teams (Levina, 2005).

Centralisation is another parameter used to characterise the relative importance of different actors in a network. It describes the extent to which the various actors in a network are equally important (i.e., central). It is a measure of individual importance (centrality) dispersion and takes the value of zero when all actors are equally important and a higher value when few or even only one actor dominates the other actors in the network (Wasserman and Faust, 1994). Centralised structures hinder cooperation (Molm, 1994) because actors need to use indirect paths, mostly through a few central actors who perform as information brokers.

SNA allows the study of networked groups as actors of higher order networks, as in a network of teams. Usually, new product development not only involves intra-team collaboration, but also collaboration with external entities, and in particular, with other project teams (Allen and Cohen, 1969; Allen, 1970; Mallick and Schroeder, 2005). The act of crossing team boundaries to seek information for the focal project has been called BSA (Allen, 1970). Previous research provides some empirical support for the argument that BSA improves overall performance and creativity yet largely hinders productivity (Allen, 1970; Dollinger, 1984; Brass *et al.*, 2004) in that it requires additional effort to seek external knowledge, and this acquired knowledge often must be reciprocated to external teams, thereby consuming resources that could otherwise be used for adding value to the focal task.

Learning rates

In the operations and information systems literature, learning refers to a sustained decrease in the effort required to produce a unit of output. Historically, the first series of academic articles concerning learning in industrial contexts was due to Wright (1936) who observed that the number of man-hours required to complete the assembly of aircraft decreased steadily, following an exponential function. In general, the learning curve, a function representing the effort per unit of output as a function of the produced volume, can be described as a negative exponential function of the form

$$Y_{(x)} = ax^{-b},$$

where

$Y_{(x)}$ is the effort (or unit cost) to produce the xth unit,
a is the cost to produce the first unit,
b is a parameter related to the learning rate L by:

$$b = -\log(L)/\log(2)$$

or

$$L = 2^{-b}$$

where L is the percentage to which the unit cost decreases after every doubling up of the cumulative production volume. For instance, $L = 0.8$ means that every time the production volume doubles up, the unit cost decreases to 80% of the original. Note that, perhaps counter intuitively, under this definition for the learning rate, more intensive learning corresponds to a smaller L, e.g., a project with $L = 0.7$ or 70% learns faster than a project with $L = 0.8$ or 80%. Note also that when log $[Y(x)]$ is regressed on $\log(x)$ the learning curve is a negative sloped straight line with slope equal to $-b$.

The practical uses of learning curves and their associated learning rates are capacity estimation, forecasting, budgeting, supplier quote evaluation, and generally any issues that require the estimation of the effort (or manpower) to produce a given number of units of output and/or unit costs (Yelle, 1979). Although learning rates by industry are more or less typical of the product being made, variance among organisations producing the same kind of product is notable and very relevant to production costs (Argote *et al.*, 1990).

Learning curves have been used extensively to forecast resources and predict production costs (Sallenave, 1985), but mostly in a context of mass goods production, and in particular assembly lines. A famous case of adherence to the exponential learning curve is that of Ford's model T automobile, where the

repetitiveness of the tasks involved and the high production volumes originated an almost perfect exponential curve until General Motors disrupted the market and new models had to be introduced (Hirschmann, 1964).

Other more complicated mathematical models than a negative exponential formula have been introduced to describe data from specific industries, but the basic exponential model holds acceptably well in most situations, avoiding the problem of estimating several additional curve parameters (Yelle, 1979).

Although software development is less repetitive than mass production, the theory of learning curves can indeed be applied to it (Reagans *et al.*, 2005). There is some empirical support for the exponential form of the learning curve holding true when applied to individual developers or to teams producing software using modern development tools (Pendharkar and Subramanian, 2007), and the learning curve effect has been observed in the software testing stage as well (Abu *et al.*, 2005).

Learning can be observed in organisations at the individual or aggregate levels, the former called labour learning and the latter, organisational learning (Argote, 2012). Factors that influence learning rates have been framed into three categories (Argote *et al.*, 2003): those related to the properties of the knowledge itself, of the organisations learning, and of the relationships between such organisations.

The first group includes task complexity (Argote *et al.*, 1995), whether the knowledge is explicit or tacit (Nadler *et al.*, 2003), the relative difficulty to code/decode knowledge (Zander and Kogut, 1995), and whether the knowledge is perceived as internal or external to the organisation (Menon and Pfeffer, 2003), or public versus private (Uzzi and Lancaster, 2003).

The second group includes expertise or "expert status" (Reagans *et al.*, 2005), worker turnover (Argote *et al.*, 1990), and previous experience in similar tasks or in different but related tasks (Reagans *et al.*, 2005).

The third group of factors refers to the team's organisation and is perhaps the most relevant for this paper. We are interested in describing structure as pertaining not only to connections among team members, but also to connections with other teams. It is interesting to note that several studies' conclusions refer, perhaps inadvertently, to aspects that are standard fare in SNA empirical studies, indicating that the subject matter is conducive to performing a large scale, hard-metrics-based empirical study that would be squarely framed in the SNA tradition rather than deriving network-related conclusions from a reductionist view.

For instance, similarity among connected learners seems to boost learning (Song *et al.*, 2003), which in SNA, would be called the homophily or "philos" effect (Krackhardt, 1992). In a study using MBA students, those teams that had a more homogeneous knowledge distribution outperformed the teams whose knowledge was concentrated in few members (Rulke and Galaskiewicz, 2000). To a researcher adept at SNA imagery, this result appears to suggest a relationship

between learning rates and network density (a dense network has ties between most pairs of actors and information is homogeneously distributed). A mathematical model predicted that centralised firms would have an advantage when competition is intense (Chang and Harrington, Jr., 2003), which in SNA terms would indicate an association between network centralisation and performance, subject to the influence of competition as a moderator. Bureaucratic teams have been found to be better learners (Bunderson and Boumgarden, 2009), which has been justified as possibly associated with communication patterns within the team. Other studies point to, and even suggest that, a relational aspect of learning has yet to be studied beyond the dyadic relationship, and the structural impacts on learning rates have been suggested as an emergent theme in organisational learning (Argote *et al.*, 2003) but to our knowledge, not yet empirically explored. All in all, the extant literature seems to indicate that network characteristics could be linked to learning rates, though no study has tested this proposition directly.

The previously mentioned papers encompassed a range of products and industries, among which software was in the minority. What are the factors that impact learning in the specific context of software development? One study, limited to one proprietary software product (Boh *et al.*, 2007) showed that previous experience is an important learning factor. Among self managed teams (Crowston *et al.*, 2007), the more specialised and hierarchical teams were found to be better learners (Bunderson and Boumgarden, 2009), and learning was improved when engaged in transferring technology from other projects (Singh *et al.*, 2011), which suggests that not only intra-team, but also inter-team links are relevant. Software development learning rates have been shown to depend on how information is processed between implicit and explicit (Zorgios *et al.*, 2009) yet still conform to a learning curve.

Although learning rates have not been explicitly linked to social structure, there is strong, yet sometimes contradictory evidence relating other potential performance metrics with some aspects of social structure such as self-assessed team performance or viability (Ahuja and Carley, 1999; Ahuja *et al.*, 2003; Balkundi and Harrison, 2006). For instance, network centralisation and density were associated with quality, and time between releases in software products (Colazo, 2010), though learning rates were not investigated. The presence of "emergent people" (more central actors in SNA parlance) was associated with software build success (Kwan *et al.*, 2012). In a similar stream of research, several studies have explored empirically "Conway's Law", that suggests a relationship between organisational structure with the organisation of the product itself or its modularity (Conway, 1968; Burton *et al.*, 2011; Kwan *et al.*, 2012; Bailey *et al.*, 2013; Santana *et al.*, 2013), but no study we are aware of studied the association between intra- and inter-team structure with learning rates.

From the reviewed research, the concept of learning curve and their learning rates applies to the specific realm of software development, and a network view of factors that influence learning rates seems to be fertile ground to explore.

Cognitive load theory

The theoretical lens through which we are attempting to link team structure with learning rates is Cognitive load theory (Sweller, 1988; Chandler and Sweller, 1991; Sweller, 1994; Kalyuga *et al.*, 1998; Sweller *et al.*, 1998; Kalyuga *et al.*, 2003; Sweller *et al.*, 2007), which has roots in work from the 1950s in Miller's cognitive science (Miller and Selfridge, 1950; Miller, 1953, 1956a,1956b).

As per cognitive science, the act of learning involves two mechanisms: schema acquisition and the automation of learned procedures (Cooper and Sweller, 1987). A schema is a cognitive construct that organises the elements of information according to the manner with which they will be processed. Newly presented information is altered so that it is congruent with schemas that condense the knowledge of the subject matter. When schemas are acquired, new information is automatically matched with these schemas, thereby, speeding up the learning. Schemas are continuously being adapted and modified with the input of new information.

Schema acquisition is limited by "working memory", which is short term, transient, and limited, as opposed to long term memory — which can be considered to be basically unlimited. Working memory is able to hold on to about seven plus/minus two "chunks" of information (Miller, 1956) that would be used for schema formation in problem solving and learning. Once schemas are acquired, they are moved into long term memory for later use.

Sweller contributed the key works on cognitive load theory (Sweller, 1988, 1994; Sweller *et al.*, 1998). He and his collaborators made some important observations: First, novice problem solvers use, in general, a technique called means-ends analysis. This technique involves a backwards working phase where the problem solver works backwards from the ultimate goal, finding sub-goals in succession until they arrive to a state where a sub-goal can be solved for, given the available information. Then, they proceed in forward fashion solving all higher level sub-goals until they solve the main, highest level goal.

Expert problem solvers do not use the means-ends technique, but rather they use previously acquired schemas which are accommodated and assimilated to the problem at hand, permitting the problem solver to conceptualise a whole set of logical equations that has, as the only unknown, the ultimate goal of the problem being solved. The experts' domain-specific knowledge resides in the form of schemas, which they readily apply to new problems by accommodation and

assimilation in a much more efficient process than means-ends analysis in which there is no learning, but only structured goal seeking.

As such, the key to expert problem solving is the acquisition of schemas. The limited cognitive capacity has to be used principally for schema acquisition and any cognitive load that is not used for that purpose should be minimised (Sweller *et al.*, 2007). The part of the total cognitive burden used to create or process schemas is called germane cognitive load, or germane load. In addition to this, there are the intrinsic cognitive load, which is exclusively associated with the difficulty of the task, and extraneous load, associated with how the information is presented to the problem solver, or with how much effort the learner has to exert to decode the information into usable form, for instance translating from a foreign language, identifying and weeding out irrelevant information, or spotting typos.

Because the intrinsic load depends on the difficulty of the task, it cannot be easily modified, and the extraneous load is what should be minimised instead. Extraneous cognitive load is related to the occurrence of negative effects that impair learning (Chandler and Sweller, 1991). One of those is the redundancy effect; for instance, presenting the same information twice in the same or different media or codifications (e.g., a diagram and a textual explanation at the same time). Another one is the split attention effect, which occurs when information is presented in two or more split pieces that require switching to and from all pieces to complete the processing of the information (e.g., part of the information is presented visually and another part in text, or from two teammates that have partial information, or who are available at different times).

Hypotheses

Density

In a dense structure, all actors have multiple collaborators and as such they keep multiple links with others. We contend that a denser structure exacerbates both the redundancy effect and the split attention effect, which should hinder learning.

When a team member collaborates on a task with more teammates, there is an increased chance of receiving redundant information since in established teams there is a significant amount of overlapping experience and skills transference (Leonard-Barton, 1992). In fact, the long known phenomenon of groupthink where all members of a team share the same or similar solutions to new problems (Janis, 1972) can be thought of in terms of shared schemas. Since expertise resides in the schemas committed to long term memory, collaborating with more teammates will increase the proportion of information that relates to already established schemas, and hence conforms redundant knowledge that will increase the extraneous

cognitive load and decrease the amount of working memory available for new schema formation, making learning less efficient.

Also, putting together the non-redundant portion of information received from several other teammates will require switching between collaborators to acquire complete information for schema formation. This split attention effect will also increase extraneous cognitive load in detriment of germane load, which is used to generate or establish new, or modify existing schemas.

The split attention effect is also related to the asynchronous nature of collaboration in software development, where different collaborators may contribute at different times, often in different time zones or different shifts (Espinosa and Carmel, 2003). Having to shift times has been shown to negatively affect communication effectiveness (Olson and Olson, 2000), and from the point of view of schema formation it constitutes an instance of split attention effect which will then hinder learning. This brings about the first hypothesis of this paper:

*H*1: *Collaboration structure density is negatively associated with learning.*

Centralisation

Centralised structures possess one or relatively few "central" actors that tend to channel information and act as brokers between other dyads of actors, connecting with most other team members, while the remaining actors will have few links to other teammates. Central actors tend to be experts (Balkundi and Kilduff, 2006) who are more skilful than the others, and hold a disproportionately high number of links to other actors.

Expert actors often suffer the expertise reversal effect (Kalyuga *et al.*, 2003). This effect is somewhat the opposite of the combined split attention and redundancy effects. For novices, the integration of information originally present in different media reduces the split attention effect and liberates part of the working memory for schema acquisition. Experts have already assimilated parts of the necessary information in the form of schemas stored in long term memory, and integrated information hinders rather than helps, because some of the integrated information is redundant for them. For instance, a study found that experienced electricians do better with graphical circuit information only, rather than with text explanations integrated in the diagrams, while novices react in the opposite manner (Kalyuga *et al.*, 1998).

A cognitive load that is germane to a novice can be extraneous to an expert (Paas *et al.*, 2004). Central actors will be bombarded with information from less skilled actors, and on the one hand, if the information is redundant, they will suffer the redundancy effect, but even if it is not redundant, they will suffer the expertise

reversal effect, hindering their learning. Also, central developers tend to be experts at more than one kind of task (i.e., software module), and need to be adept at managing the interfaces between modules (Balkundi and Harrison, 2006; Balkundi and Kilduff, 2006). Managing interfaces between modules, in addition to the contents of the modules themselves, increases the degree of information interactivity (Kalyuga *et al.*, 2003) and requires the acquisition of more complex schemas that include the schematisation of the interfaces of interacting information, creating a bigger load for working memory and potentially exacerbating the negative effects of extraneous load, causing a learning failure.

Given the arguments above, it follows that at any given density, a more centralised network will exhibit a higher degree of these deleterious effects and we can posit:

*H*2: *Collaboration structure centralisation is negatively associated with learning.*

Boundary spanning activity

In new product development the aggregate knowledge of the team may not be much different than what any one member knows (Leonard-Barton, 1992), which often times requires seeking information from sources external to the team, perhaps other project teams or other external actors. Further, innovation often occurs under conditions of environmental uncertainty. The knowledge required to perform development tasks can change rapidly and may not be available from existing team members. In such situations, knowledge must be sought from outside of the team, since a lack of external information may then translate into a knowledge gap between what is known and what the team needs to know in order to effectively cope with design features and changes in the competitive environment (Levitt and March, 1988; Leonard-Barton, 1992; Teigland and Wasko, 2003).

Moreover, teams that are isolated from their context exacerbate groupthink, developing a homogeneous set of beliefs that hinders creativity and penalises critical thinking (Janis, 1972). From the cognitive load perspective, groupthinkers can be thought of as having an increased cognitive bias that stems from trying to fit new problems into their already established schemas, even when these schemas are not conducive to solving the new problem.

The cognitive bias effect hinders the formation and adaptation of useful schemas and conversely, groups that access new ideas and technologies can transfer them to their tasks in the focal team, thereby forming new schemas and improving learning. Communicating with other teams can help team members gather indirectly useful information about competitive products, and stay ahead of technical developments implemented elsewhere, etc.

There is a potential negative for BSA as well: A high level of BSA can place an additional overhead on the individuals who perform boundary spanning roles, particularly in terms of time spent on sense-making and decoding of information from external to internal meaning frameworks when they are different. Here we are assuming that the possible individual productivity hindrance to the developers that sustain BSA is overwhelmed by BSA's positive effect for the team in helping create and adapt schemas, and the third hypothesis is:

*H*3: *BSA is positively associated with learning.*

Complexity

Product complexity is defined as the number of parts and modules necessary to produce a product (Clark and Fujimoto, 1991) and to understand it, it is necessary not only to know which parts make a complete product but also how those parts interact with each other (Ruthen, 1993). In general, empirical studies on new product development performance attempt to control for product complexity (Hobday, 1998; McDermott, 1999).

From the cognitive load theory point of view, task complexity is directly related to the intrinsic portion of cognitive load (Van Merrienboer and Sweller, 2005) since the latter deals with the difficulty in understanding how the product operates, which is related to its complexity (Robinson, 2001).

Learning tasks with a small intrinsic cognitive load may not reach the operative limit of the working memory. Only when working memory is crowded by extraneous load, there will be a limiting effect on learning. The more complex the task, the more likely it is to present both the redundancy effect and the split attention effect typical of the formation of extraneous cognitive load. Different parts of the product may operate using the same or similar mechanisms (for instance an IF-THEN loop in software presents a feedback mechanism similar to the one in a WHILE loop) presenting redundancy to the learning and generating extraneous cognitive load. Also the more different parts in a product the more exacerbated the split attention effect.

In other words, the more complex the task, the more pronounced the effects of extraneous load will be, and the impact of structural parameters on learning is going to be affected more strongly. This argues for a moderating effect of task complexity on the precedent hypotheses:

*H*4(a, b, c): *Task complexity moderates the association between learning and* (a) *density*, (b) *centralisation and* (c) *BSA.*

The overall research model can be seen in Fig. 1.

Methods

Research framework

In order to empirically test the hypotheses, the selected research framework should offer rich, hard data on collaboration structure, productivity, and task complexity in a new software development scenario. OSS development teams fit these requirements nicely.

OSS not only relies on making public the source code for the product under development, but also normally produces other artefacts that can be mined for data, such as mailing lists, change logs, and so on, providing great opportunities for collecting the data needed for this study.

OSS project team information is hosted in web-based repositories that have been used repeatedly as sources of archival data for empirical studies. In accordance with the majority of previous OSS empirical studies (e.g., Hahn *et al.*, 2008; Colazo and Fang, 2009), this research uses data collected from OSS projects hosted in Source Forge (SF) (www.sourceforge.net).

Sampling

The OSS project data were collected initially in early 2008, with updates to the database throughout mid 2013. Among all OSS projects hosted in SF, the projects analysed were restricted to those using "C" as their programming language. Constraining the sample to a single programming language is strongly preferred when using code-based metrics (Jones, 1986) such as productivity and "C" is a procedural language for which there are well-established metrics that are relatively easier to collect and crosscheck than if an object-oriented language had been used, in which case, other aspects such as the need to consider coupling and cohesion would make data collection and analysis much more resource intensive and difficult to compare among languages. A program was considered written in "C" when at least 90% of their source code files were exclusively in that language.

In SF, an overwhelming majority of registered projects do not have any meaningful activity, with no source code at all, or are single developer projects (Krishnamurthy, 2002). Following previous empirical research on OSS teams, projects with six or more core team members were selected (Crowston and Howison, 2003; Colazo and Fang, 2010). Core team members are defined here as project members who have administrative rights to write source code in the repository (Mockus *et al.*, 2002).

In the SF repository, there were 587 software projects being developed by six or more core team members and using "C". These projects constituted the sampling frame. The final sample was reduced to 230 projects (39% of the sampling frame)

because only that number had archived full data on all development activities. In spite of the non-probabilistic nature of the sample (see limitations), it contained projects with varied types of applications, sizes, and degrees of complexity. The unit of analysis was the OSS project.

Measurement

Collaboration structure

Collaboration structure was measured following metrics common in SNA (Wasserman and Faust, 1994). Two developers collaborate when they work on the same file. In the resulting network, developers are the actors and there is a link, or arc, when two developers collaborate. As mentioned before, the collaboration structure within the team is characterised by two main parameters: density and centralisation. Collaboration with other teams is captured by BSA, in a second-order network where OSS projects are the actors and there is a link between two projects if two developers from each of the projects collaborate.

The network data were obtained from the activity logs of each project's Subversion, a standard tool that registers each modification made by a team member to any file belonging to the project; it is used to prevent programming conflicts in multi-member projects. The log files contain, among other information, the name of the file modified, the name of the team member making the modification, the kind of modification made, the number of lines added, lines deleted, and lines modified.

A custom-made script downloaded these logs and extracted the detail of who worked on each file. The number of team members was calculated from the logs as the number of different individuals who made changes to the project's files.

Another script translated the Subversion log information into associative data relating each possible team member dyad to the files they worked on in common, if any. These network data were fed to the SNA module of "R" (R Development Team, 2014) to obtain the structural parameters (network density, network centralisation and BSA).

Density

A graph is defined as a collection of nodes and arcs linking them. In a graph, density is defined as the ratio of the number of arcs to the maximum possible number of arcs. In a valued graph (one in which arcs have a value or "strength"), this concept is extended by assigning to each arc its value and to each missing arc the value of 0, density's value coincides with the arithmetic average strength of the arcs.

Centralisation

There are several measures of the centralisation of a social structure, such as, degree centralisation, betweenness centralisation, information centralisation, etc. (Wasserman and Faust, 1994). All centralisation measures capture the variance of the individual actor's centralities or the ratio between the network's average centrality and the maximum possible theoretical average centrality given the specific topology of the network (Freeman, 1977). Here we use Freeman's version of centralisation, where degree centrality is the number of links per actor (Wasserman and Faust, 1994).

Boundary spanning activity

BSA was measured as the developer-based, inter-team degree centrality, normalised by team size (Wasserman and Faust, 1994). For instance, if the focal team has 10 members, and five of them have at least one link each to developers in other project teams, the BSA will be $5/10 = 0.5$. The more individuals link to other teams' developers, the higher the BSA.

Learning rate

First, productivity was measured as the number of lines of code written per developer per month, a popular and established measure for development productivity (cf. Jones, 1986).

Then, the learning rate was calculated as the slope of the log–log linear regression approximation where the dependent variable was productivity and the independent variable was elapsed time since the first line of code was committed to the repository.

Complexity

Basili and Hutchens (1983) define software complexity as a measure of the resources expended while interacting with software code. This complexity arises from the organisation of programming elements within the software code.

When software is more complex, more information needs to be processed during coding by developers. When they collaborate on more complex software, developers must not only comprehend each of the individual modules, but also spend additional cognitive effort to understand how different modules and paths are interrelated, making collaboration more demanding. A software artefact is more complex when it contains more interrelated modules or paths (Darcy *et al.*, 2005). A widely accepted measure for software complexity is McCabe's cyclomatic complexity, which measures the intricacy of the logic flow in the program (McCabe, 1976). That is the measure used in this paper.

Other variables/controls

The sampling design controlled for programming language to eliminate potential inconsistencies in the metrics based on source code. Project tenure was measured by counting the number of days between the first known date of activity in the source code repository and the date the measurements were taken. Project size was measured in total lines of source code. The number of developers was taken from the Subversion logs.

Analysis and Results

Unexpectedly, we found that overall there was little learning happening. The overall sample learning rate was 0.989; i.e., when the elapsed time doubled up it took only 1.1% less effort to commit one additional KSLOC. This result is far from the reported 80% learning rate estimated for software development (Raccoon, 1996). However, on further inspection, we observed that while teams on some projects learned (productivity increased over time), some others "unlearned", i.e., their productivity became progressively worse. About 20% of the projects' teams fell into the first category and the remaining, in the second (Table 1).

Looking at the teams that learned, however, the average positive learning rate was 78%, which is well in line with the cited estimate (Raccoon, 1996). While the teams that worsened over time had an average of 79% decrease of productivity every time the code base doubled up (a "negative" 21% learning rate).

Given the observations above, we split the sample into two subgroups: the first, with positive learning, and the second with negative learning, or "unlearning", to run the regression models. Bivariate correlations of the transformed variables for the OLS regressions are shown in Table 2, while the regression results are shown in Tables 3 and 4.

For the subgroup with positive learning, collaboration network density was positively associated with learning but the cross product of density × complexity was not statistically significant, yielding support to H1 but not to H4a. While the learning rate L increases with density (the project learns less with increased

Table 1. Learning rates.

	% Sample	Mean (L)
Overall	100	0.011
Positive	20	0.78
Negative	80	1.21

Table 2. Bivariate correlations.

		1	2	3	4	5	6	7	8	9	10	11
1	Learning rate, slope	1	−0.045	−0.009	0.034	−0.291**	−1.780*	0.155*	−0.331**	−0.053	−0.049	0.147*
2	Team size, core developers		1	0.094	0.294**	0.120*	0.425***	0.055	−0.065	0.067	0.176	−0.005
3	Project tenure			1	0.306**	0.141*	0.154	0.291*	0.237*	0.173	0.079	0.307**
4	Product size, KSLOC				1	0.051	0.118	0.079	0.116	0.030	−0.012	0.088
5	Density					1	0.414**	−0.079	0.084	0.894***	0.47**	0.008
6	Centralisation						1	−0.053	0.052	0.354**	0.741***	0.020
7	BSA							1	0.062	−0.066	−0.105	0.795***
8	Complexity							1	1	0.427***	0.159*	0.426***
9	Complexity × Density									1	0.488**	0.183
10	Complexity × Centralisation										1	0.001
11	Complexity × BSA											1
	Mean	1.95	0.90	−0.10	2.25	0.65	1.02	1.00	0.96	0.63	4.57	10.35
	SD	20.67	0.10	0.09	0.53	0.47	1.40	0.28	0.31	0.54	10.91	7.29

Note: $N = 230$, *$p < 0.05$, **$p < 0.01$, ***$p < 0.001$.

Table 3. Regression, positive learning rates.

	Coeff.	Sig.
(Constant)	−0.424	***
Team size	−1.878	
Tenure	−11.345	
Product size	−1.255	*
Density	2.007	***
Centralisation	−1.655	*
BSA	−2.009	
Complexity	6.472	***
Complexity × Density	−3.320	
Complexity × Centralisation	0.140	**
Complexity × BSA	−0.689	***

Note: Dependent variable: (L) Learning rate, $N = 47$, $R^2 = 0.11$, $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

density), product complexity does not moderate such relationship. In the subgroup with negative learning, similar results were obtained.

Looking at centralisation in the subgroup with positive learning, both the main effect of centralisation and the cross product between centralisation and complexity were significantly associated to the learning rate, supporting H2 and H4b. The positive coefficient of the interaction term indicates that when complexity increases, there is a positive addition to the influence of centralisation in learning rate, i.e., teams for projects with more complex products learn less when they

Table 4. Regression, no (negative) learning.

	Coeff.	Sig.
(Constant)	0.005	
Team size	−0.952	
Tenure	−1.441	
Product size	−1.114	*
Density	1.980	***
Centralisation	−0.885	
BSA	3.08	*
Complexity	4.877	***
Complexity × Density	−0.333	
Complexity × Centralisation	0.008	
Complexity × BSA	0.001	

Note: Dependent variable: (L), $N = 183$, $R^2 = 0.36$, $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$.

Table 5. Summary of hypotheses.

Hyp.	Description	Subgroup	
		Learning	Unlearning
H1	Higher density –> less learning	**Significant**	**Significant**
H2	Higher centralisation –> less learning	**Significant**	Not significant
H3	Higher BSA –> more learning	**Significant**	**Significant in opposite direction**
H4a	Complexity interacts with density	Not significant	Not significant
H4b	Complexity interacts with centralisation	**Significant**	Not significant
H4c	Complexity interacts with BSA	**Significant**	Not significant

become more centralised. The average main effect of centralisation cannot be meaningfully explained in the presence of an interaction term, and we can only say that our sample exhibited, on average, a beneficial influence (negative coefficient) of centralisation that deteriorates with increasing complexity. In the "unlearning" subgroup, neither the interaction nor the main effect was significant for centralisation.

In the matter of BSA, the positive learning group exhibited both a significant main effect and interaction effect between complexity and BSA, supporting H3 and H4c on learning rates. On average, the effect of BSA is beneficial (negative coefficient), and this effect significantly increases when the product becomes more complex. The unlearning group showed contrasting results: there was no significant interaction effect with complexity, but the coefficient of the main effect is significantly positive in sign, i.e., the more BSA, the more the focal project team unlearns, independently of complexity. Table 5 shows a summary of empirical results.

Discussion

Based on previous research we were expecting to see an overall positive learning rate of about 80% (Raccoon, 1996), but overall, in our sample there was practically no learning. Only when we split the sample into subgroups, 20% of the projects displayed the expected average 78% positive learning, while the remainder of the sample showed negative learning rates, or "unlearning". The existence of software development teams that "unlearn" has been previously documented (Lyytinen and Robey, 1999), but based on few, albeit resounding cases of business applications and not on a broader sample of varied products. We do not know if the proportions observed in this study are generalisable to proprietary software development, but these results raise increased awareness of the problem of unlearning in software development and should spur further inquiry, especially because the consequences

of unlearning in new product development in general are still not well understood (Akgün *et al.*, 2006). Perhaps as important as finding how software development teams learn, is finding how many project teams embark in a trajectory of constant performance deterioration. Given that the hypotheses derived in this paper seemed to apply nicely to the positive learning subgroup, but not to the unlearning subgroup, there is a possibility for different theoretical mechanisms that could explain learning *vis-à-vis* unlearning in software development. Given that unlearning, though detrimental to productivity, may present some beneficial side effects such as stimulating the capacity for improvisation (Akgün *et al.*, 2007), future studies should attempt to measure not only productivity but also other performance metrics simultaneously.

In all cases, whether there is positive or negative learning, increased collaboration density reduces learning rates, and the effect of density is independent of product complexity. Cognitive load theory explains this observation in terms of two main effects, redundancy of knowledge and split attention.

Practically, project managers would do well in limiting the number of team members that work in any given file or module, as well as trying to pre-select those collaborating team members in such a way that their expertise do not overlap much. Another interesting practical application could be the flagging of developers who want to collaborate in a given file or module. Some projects flag defects (Colazo, 2014), classifying them into categories to allow developers to self-select in accordance to the expertise needed to solve that particular kind of problem (Dalle and den Besten, 2007).

Something similar could be done by, instead of flagging bugs, projects could publicly flag developers' skills, to warn project managers when several developers with overlapping skills are already collaborating in a given module and prevent additional knowledge redundancy. This would be particularly applicable in OSS, where developers in principle, assign themselves to programming tasks rather than being assigned by managers. Another technique to decrease the split attention effect could be trying to minimise time dispersion of the development team (Carmel and Espinosa, 2010).

For the learning subgroup, centralisation interacts with complexity, i.e., teams of projects with more complex products learn less when they become more centralised. Cognitive load theory explains this impact as a result of: (1) the expertise reversal effect, and (2) the additional extraneous load associated with not only managing contents but also interfaces between different kinds of schemas (Kalyuga *et al.*, 2003). Furthermore, both effects logically become more important when the product is more complex.

The coefficient for the main effect of centralisation was negative while the one for the interaction term was positive; indicating that at the average product

complexity found in the sample, an increase in centralisation makes the team learn at a slower rate. The implication of this result is, project managers should try to keep centralisation in check in relatively more complex products, or try to decentralise the collaboration structure when products are very complex. For the unlearning subgroup, neither interaction nor the main effect of centralisation was significant and this characteristic of the collaboration structure does not seem to be related to their learning rates.

Complexity interacted with BSA only in the positive learning group, and the main effect's sign is consistent with a positive influence of BSA on learning rate. That is, at least, on average and in the sample explored. The effect of BSA seems to be more beneficial when the product becomes more complex, supporting that outside knowledge searching should be especially encouraged when developing complex software. In the unlearning group, the interaction was not significant but there is a main effect of the opposite sign, and BSA seems to hinder learning. A possible explanation could be that developers who work in focal projects that exhibit positive learning, bring in information that promotes schema formation and in the end is useful for speeding up productivity, while in projects that typically "unlearn", the boundary spanning developers may be directing their focus towards the external projects and neglecting the focal one, which we cannot ascertain with the data at hand but should be explored in more detail in future research using data related to developers who work on a portfolio of projects simultaneously (Cooper *et al.*, 2001).

While cognitive load theory seems to acceptably explain how collaboration structure is related to learning rates in teams that indeed have positive learning rates, it seems to have shortcomings in explaining why other teams unlearn. Several of the structural associations described above, that are significant in learning teams, do not apply to unlearning teams, suggesting that mechanisms that do not directly involve a structural influence are at play, warranting further research into these teams, where performance deteriorates steadily over time.

Limitations and Future Research

Empirical work is never absent of limitations. First it should be noted that this study relied solely on archival data and that the sample was non-probabilistic, and refers only to relatively larger software development teams with six or more developers. Although this limitation prevents generalising results to smaller teams, in practice, bigger teams are associated with products that are likely to become popular and have bigger user bases and be more relevant for business. Another limitation is that the sample included only OSS projects, and results should be replicated in a proprietary environment before freely generalising conclusions to

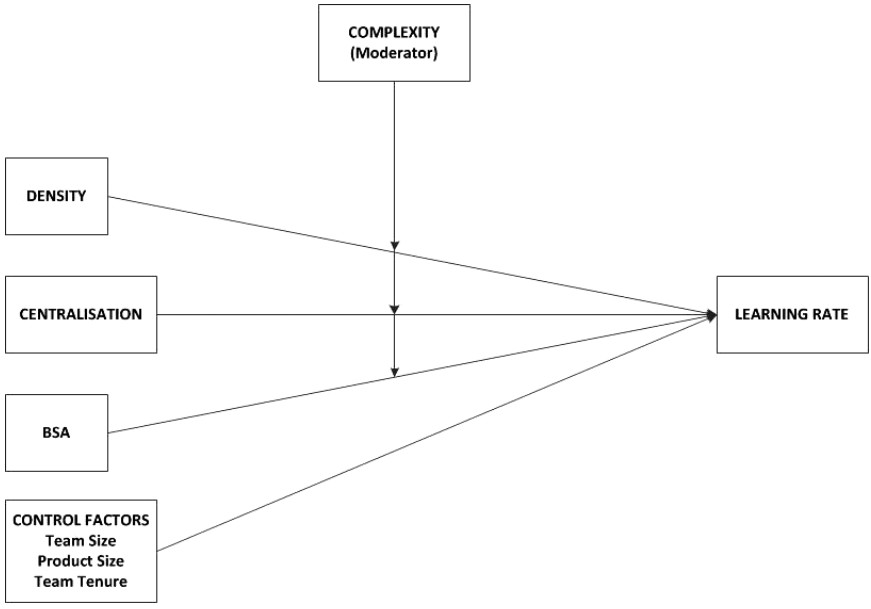

Fig. 1. Research Model.

commercial software development. Third, the sample used projects developed only in a procedural programming language and given the importance of object-oriented languages the results should be confirmed in projects dealing with languages such as Java or C++.

A logical new research question stemming from these results is: What makes development teams either learn or unlearn? While our hypotheses were based on the assumption that all teams have positive learning, the empirical data showed that there are two clearly different subgroups of projects, and while it is important to understand what makes learning more effective, it is also important to understand why other teams degrade their performance over time, and the factors related to their collaboration structures.

The study should be replicated in a proprietary software development. It would be interesting to explore how the proportions between learning teams and unlearning teams vary depending on contextual factors.

References

Abu, G, JW Cangussu and J Turi (2005). A quantitative learning model for software test process. In *Proc. 38th Annual Hawaii Int. Conf. System Sciences, 2005, HICSS'05*, p. 786. New York: IEEE.

Ahuja, MK and KM Carley (1999). Network structure in virtual organizations. *Organization Science*, 10(6), 741–757.

Ahuja, MK, DF Galletta and KM Carley (2003). Individual centrality and performance in virtual R&D groups: An empirical study. *Management Science*, 49(1), 21–38.

Akgün, AE, JC Byrne, GS Lynn and H Keskin (2007). New product development in turbulent environments: Impact of improvisation and unlearning on new product performance. *Journal of Engineering and Technology Management*, 24(3), 203–230.

Akgün, AE, GS Lynn and JC Byrne (2006). Antecedents and consequences of unlearning in new product development teams. *Journal of Product Innovation Management*, 23 (1), 73–88.

Allen, TJ (1970). Communications networks in R&D laboratories. *R&D Management*, 1(1), 14–21.

Allen, TJ and S Cohen (1969). Information flow in R&D labs. *Administrative Science Quarterly*, 14(1), 12–19.

Argote, L (2012). *Organizational Learning: Creating, Retaining and Transferring Knowledge*, Berlin: Springer.

Argote, L and D Epple (1990). Learning curves in manufacturing. *Science*, 247(4945), 920–924.

Argote, L, B McEvily and R Reagans (2003). Managing knowledge in organizations: An integrative framework and review of emerging themes. *Management Science*, 49(4), 571–582.

Argote, L, SL Beckman and D Epple (1990). The persistence and transfer of learning in industrial settings. *Management Science*, 36(2), 140–154.

Argote, L, CA Insko, N Yovetich and AA Romero (1995). Group learning curves: The effects of turnover and task complexity on group performance. *Journal of Applied Social Psychology*, 25(6), 512–529.

Bailey, SE, SS Godbole, CD Knutson and JL Krein (2013). A decade of Conway's law: A literature review from 2003–2012. In *3rd Int. Workshop on Replication in Empirical Software Engineering Research (RESER)*, 2013, New York: IEEE.

Balkundi, P and DA Harrison (2006a). Ties, leaders and time in teams: Strong inference about network structure's effects on team viability and performance. *Academy of Management Journal*, 49(1), 49–68.

Balkundi, P and M Kilduff (2006b). The ties that lead: A social network approach to leadership. *The Leadership Quarterly*, 17(4), 419–439.

Basili, V and D Hutchens (1983). An empirical study of a syntactic complexity family. *IEEE Transactions on Software Engineering, SE* 9(6), 664–672.

Boh, WF, SA Slaughter and JA Espinosa (2007). Learning from experience in software development: A multilevel analysis. *Management Science*, 53(8), 1315–1331.

Brass, DJ, J Galaskiewicz, R Greve and W Tsai (2004). Taking stock of networks and organizations: A multilevel perspective. *Academy of Management Journal*, 47(6), 795–817.

Bunderson, JS and P Boumgarden (2009). Struture and learning in self-managed teams: Why "bureaucratic" teams can be better learners. *Organization Science*, 21(3), 609–624.

Burton, SH, PM Bodily, RG Morris, CD Knutson and JL Krein (2011). Design team perception of development team composition: Implications for Conway's law. In *2nd Int. Workshop on Replication in Empirical Software Engineering Research (RESER)*, 2011, pp. 52–60. New York: IEEE.

Carmel, E and JA Espinosa (2010). *I'm Working While they're Sleep: Time Zone Separation Challenges and Solutions*. USA: Nedder Stream Press.

Chandler, P and J Sweller (1991). Cognitive load theory and the format of instruction. *Cognition and Instruction*, 8(4), 293–332.

Chang, M-H and JE Harrington, Jr. (2003). Multimarket competition, consumer search, and the organizational structure of multiunit firms. *Management Science*, 49(4), 541–552.

Chidambaran, L and LL Tung (2005). Is out of sight, out of mind? An empirical study of social loafing in technology-supported groups. *Information Systems Research*, 16(2), 149–168.

Clark, KB and T Fujimoto (1991). *Product Development Performance*. Boston: Harvard Business School Press.

Colazo, JA (2010). Collaboration structure and performance in new software development: Findings from the study of open source projects. *International Journal of Innovation Management*, 14(5), 735–758.

Colazo, J (2014). Performance implications of stage-wise lead user participation in software development problem solving. *Decision Support Systems*, 67, 100–108.

Colazo, JA and Y Fang (2009). Impact of license choice of open source software development activity. *Journal of the American Society for Information Science and Technology*, 60(5), 997–1011.

Colazo, JA and Y Fang (2010). Following the sun: Temporal dispersion and performance in open source software project teams. *Journal of the Association for Information Systems*, 11(11), 684–707.

Conway, ME (1968). How do committees invent. *Datamation*, 14(4), 28–31.

Cooper, G and J Sweller (1987). Effects of schema acquisition and rule automation on mathematical problem-solving transfer. *Journal of Educational Psychology*, 79(4), 347.

Cooper, RG, EJ Kleinschmidt and SJ Edgett (2001). *Portfolio Management for New Products*. Cambridge, MA: Perseus Books.

Crowston, K and J Howison (2003). The social structure of free and open source software development. *First Monday*, 10(2).

Crowston, K, Q Li, K Wei, UY Eseryei and J Howison (2007). Self-organization of teams for free/libre opensource software development. *Information and Software Technology*, 49(6), 564–575.

Dalle, J-M and M den Besten (2007). Different bug fixing regimes? A preliminary case for superbugs. In *Open Source Development, Adoption and Innovation*, Berlin: Springer, pp. 247–252.

Darcy, DP, CF Kemerer, SA Slaughter and JE Tomayko (2005). The structural complexity of software: An experimental test. *IEEE Transactions on Software Engineering*, 31, 11.

Davern, M, T Shaft and D Te'eni (2012). Cognition matters: Enduring questions in cognitive IS research. *Journal of the Association for Information Systems*, 13(4), 273–314.

de Santana, AM, FQ da Silva, RC de Miranda, AA Mascaro, TB Gouveia, CV Monteiro and AL Santos (2013). Relationships between communication structure and software architecture: An empirical investigation of the Conway's law at the Federal University of Pernambuco. In *23rd Int. Workshop on Replication in Empirical Software Engineering Research (RESER), 2013*. New York: IEEE.

Dollinger, MJ (1984). Environmental boundary spanning and information processing effects on organizational performance. *The Academy of Management Journal*, 27(2), 351–368.

Dwivedi, YK, D Wastell, S Laumer, HZ Henriksen, MD Myers, D Bunker, A Elbanna, M Ravishankar and SC Srivastava (2014). Research on information systems failures and successes: Status update and future directions. *Information Systems Frontiers*, 17, 1–15.

Espinosa, JA and E Carmel (2003). The impact of time separation on coordination in global software teams: A conceptual foundation. *Software Process Improvement and Practice*, 8, 249–266.

Freeman, LC (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1), 35–41.

Glass, RL (2005). IT failure rates: 70% or 10–15%? *IEEE Software*, May–June 2005, 112–114.

Hahn, J, JY Moon and C Zhang (2008). Emergence of new project teams from open source software developer networks: Impact of prior collaboration ties. *Information Systems Research*, 19(3), 369–391.

Hirschmann, WB (1964). Profit from the learning-curve. *Harvard Business Review*, 42(1), 125–139.

Hobday, M (1998). Product complexity, innovation and industrial organisation. *Research Policy*, 26(6), 689–710.

Janis, IL (1972). *Victims of Groupthink*. Boston, MA: Houghton Mifflin.

Jones, C (1986). *Programming Productivity*. New York: McGraw-Hill.

Kalyuga, S, P Chandler and J Sweller (1998). Levels of expertise and instructional design. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, 40(1), 1–17.

Kalyuga, S, P Ayres, P Chandler and J Sweller (2003). The expertise reversal effect. *Educational Psychologist*, 38(1), 23–31.

Krackhardt, D (1992). The strength of strong ties: The importance of philos in organizations. *Networks and organizations: Structure, Form, and Action*, 216, 239.

Krishnamurthy, S (2002). Cave or community? An empirical examination of 100 mature open source projects. *First Monday*, 7(16).

Kwan, I, M Cataldo and D Damian (2012). Conway's law revisited: The evidence for a task-based perspective. *IEEE Software*, 29(1), 90–93.

Leonard-Barton, D (1992). Core capabilities and core rigidities: A paradox in managing new product development. *Strategic Management Journal*, 13 (Special Issue: Strategy Process: Managing Corporate Self-Renewal), 111–125.

Levina, N (2005). Collaborating on multiparty information systems development projects: A collective reflection-in-action view. *Information Systems Research*, 16(2), 109–130.

Levitt, B and JG March (1988). Organizational learning. *Annual Review of Sociology*, 14 (8), 319–340.

Lyytinen, K and D Robey (1999). Learning failure in information systems development. *Information Systems Journal*, 9(2), 85–101.

Mallick, DN and RG Schroeder (2005). An integrated framework for measuring product development performance in high technology industries. *Production and Operations Management*, 14(2), 142.

McCabe, T (1976). A software complexity measure. *IEEE Transactions on Software Engineering, SE* 2(4), 308–320.

McDermott, CM (1999). Managing radical product development in large manufacturing firms: A longitudinal study. *Journal of Operations Management*, 17(6), 631–644.

Menon, T and J Pfeffer (2003). Valuing internal vs. external knowledge: Explaining the preference for outsiders. *Management Science*, 49(4), 497–513.

Miller, GA (1953). What is information measurement? *American Psychologist*, 8(1), 3.

Miller, GA (1956a). Human memory and the storage of information. *IRE Transactions on Information Theory*, 2(3), 129–137.

Miller, GA (1956b). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological review*, 63(2), 81.

Miller, GA and JA Selfridge (1950). Verbal context and the recall of meaningful material. *The American Journal of Psychology*, 176–185.

Mockus, A, RT Fielding and J Herbsleb (2002). Two case studies of open source software development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology*, 11(3), 309–346.

Mohr, LB (1982). *Explaining Organizational Behavior*. San Francisco: Jossey-Bass

Molm, LD (1994). Dependence and risk: Transforming and structure of social research. *Social Psychology Quarterly*, 57(3), 163–176.

Nadler, J, L Thompson and LV Boven (2003). Learning negotiation skills: Four models of knowledge creation and transfer. *Management Science*, 49(4), 529–540.

Oh, H, MH Chung and G Labianca (2004). Group social capital and group effectiveness: The role of informal socializing ties. *Academy of Management Journal*, 47(6), 860–875.

Olson, GM and JS Olson (2000). Distance matters. *Human-Computer Interaction*, 15(2), 139–178.

Paas, F, A Renkl and J Sweller (2004). Cognitive load theory: Instructional implications of the interaction between information structures and cognitive architecture. *Instructional science*, 32(1), 1–8.

Pendharkar, PC and GH Subramanian (2007). An empirical study of ICASE learning curves and probability bounds for software development effort. *European Journal of Operational Research*, 183(3), 1086–1096.

R Development Team, The R Project for Statistical Computing: R V 3.1.1.2014.

Raccoon, L (1996). A learning curve primer for software engineers. *ACM SIGSOFT Software Engineering Notes*, 21(1), 77–86.

Reagans, R and EW Zuckerman (2001). Networks, diversity and productivity: The social capital of corporate R&D teams. *Organization Science*, 12(4), 502–517.

Reagans, R, L Argote and D Brooks (2005). Individual experience and experience working together: Predicting learning rates from knowing who knows what and knowing how to work together. *Management Science*, 51(6), 869–891.

Robinson, P (2001). Task complexity, task difficulty, and task production: Exploring interactions in a componential framework. *Applied Linguistics*, 22(1), 27–57.

Rulke, DL and J Galaskiewicz (2000). Distribution of knowledge, group network structure, and group performance. *Management Science*, 46(5), 612–625.

Ruthen, R (1993). Adapting to complexity. *Scientific American*, 268(1), 130–140.

Sallenave, J-P (1985). The uses and abuses of experience curves. *Long Range Planning*, 18(1), 64–72.

Singh, PV, Y Tan and N Young (2011). A hidden Markov model of developer learning dynamics in open source software projects. *Information Systems Research*, 22(4), 790–807.

Song, J, P Almeida and G Wu (2003). Learning-by-hiring: When is mobility more likely to facilitate interfirm knowledge transfer? *Management Science*, 49(4), 351–365.

Stepanek, G (2011). *Programming Secrets: Why Software Projects Fail*. New York: Apress.

Sweller, J (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.

Sweller, J (1994). Cognitive load theory, learning difficulty, and instructional design. *Learning and instruction*, 4(4), 295–312.

Sweller, J, PA Kirschner and RE Clark (2007). Why minimally guided teaching techniques do not work: A reply to commentaries. *Educational Psychologist*, 42(2), 115–121.

Sweller, J, JJ Van Merrienboer and FG Paas (1998). Cognitive architecture and instructional design. *Educational Psychology Review*, 10(3), 251–296.

Teigland, R and MM Wasko (2003). Integrating knowledge through information trading: Examining the relationship between boundary spanning communication and individual performance. *Decision Sciences*, 34(2), 261–286.

Uzzi, B and R Lancaster (2003). Relational embeddedness and learning: The case of bank loan managers and their clients. *Management Science*, 49(4), 383–399.

Van Merrienboer, JJ and J Sweller (2005). Cognitive load theory and complex learning: Recent developments and future directions. *Educational Psychology Review*, 17(2), 147–177.

Wasserman, S and K Faust (1994). *Social Network Analysis: Methods and Applications*. Cambridge, UK: Cambridge University Press.

Wright, TP (1936). Factors affecting the cost of airplanes. *Journal of Aeronautical Sciences*, 3(4), 122–128.

Yelle, LE (1979). The learning curve: Historical review and comprehensive survey. *Decision Sciences*, 10(2), 302–328.

Zander, U and B Kogut (1995). Knowledge and the speed of the transfer and imitation of organizational capabilities: An empirical test. *Organization Science*, 6(1), 76–92.

Zorgios, Y, O Vlismas and G Venieris (2009). The SECI model and the learning curve phenomenon. In *European Conference on Intellectual Capital*, Academic Conferences & Publishing International Ltd., Haarlem, The Netherlands, 589–599.